DreamFinder — Build a New Retailer Runbook

> *Internal guide. Walks through spinning up a new retailer deployment from a completed onboarding spreadsheet and image folder. Aim for ~30 minutes per retailer once you've done it twice.*

Phase 0 — Pre-flight check

Before you start, confirm the retailer has actually delivered:

- [ ] Completed onboarding workbook (`.xlsx`) — every required field filled (the template ships **blank**; the Bel fixture is the worked example)
- [ ] Mattress images (one per mattress) in any common format — matched by the lowercased mattress **name** (e.g. `Athena` -> `athena.jpg`)
- [ ] Accessory images (one per accessory) in any common format — matched by the **Image File Name** in the Accessories tab
- [ ] Logo PNG (transparent background, ≥400px wide)
- [ ] One square app-icon PNG (≥512×512) — **optional**; the pipeline generates the 192/512 (and 180px apple-touch) sizes from this single source. You do **not** need to supply 192 and 512 separately.
- [ ] Brand logos (one per row in the Brands tab)

If anything's missing, push back to the retailer before you start. Half-built deployments are a tar pit.

Phase 1 — Repo setup

You have two patterns:

Pattern A (recommended): new repo per retailer. The canonical Bel repo doubles as the template. For each new retailer:

1. Create a new GitHub repo under their preferred org name (or yours). Example: `acmemattress/DreamFinder` .
2. Clone the Bel repo locally: `git clone https://github.com/beford782/DreamFinder.git acme-dreamfinder cd acme-dreamfinder`
3. Repoint origin at the new repo: `git remote set-url origin https://github.com/acmemattress/DreamFinder.git`

DreamFinder Build Runbook (Internal)
Page 1 of 15

Pattern B: branch in a shared monorepo. Less recommended — the white-label strings are easier to mix up across branches.

Phase 2 — Run the data converter

Drop the retailer's completed workbook and their image folder into a working directory. The image folder has `mattresses/` and `accessories/` subdirs (any common format — jpg/jpeg/png/webp; the converter normalizes them):

```
incoming/
  Acme_Store_Data.xlsx
  images/
    mattresses/    *.jpg / *.png / *.webp   (any format/size; matched by mattress NAME)
    accessories/   *.jpg / *.png / *.webp   (matched by the Image File Name column)
    brands/        *.jpg / *.png / *.webp   (one per brand; matched by the Logo File Name
column)
    logos/         <app-icon>.png           (optional; one square PNG >=512 for PWA icons)
```

From the cloned repo root:

```
python tools/convert_store_data.py incoming/Acme_Store_Data.xlsx \
  --output-dir . --source-images incoming/images
```

The converter emits the full data bundle under the repo:

- `data/mattresses.csv` (+ `data/mattresses-es.csv` when the workbook has Spanish mattress copy)
- `data/store-config.json` (store identity, colors, copy, discount, **allowedHosts**)
- `data/accessories.json`
- `data/allowed-hosts.js` (the M1 domain-lock allowlist — see Phase 3)
- `manifest.json` (PWA metadata)
- normalized **JPG** product images into `images/mattresses/` and `images/accessories/`
- **brand logos copied verbatim** into `images/brands/` (one per brand with a Logo File Name)
- **PWA icons** `icon-192.png`, `icon-512.png`, and `apple-touch-icon.png` at the repo root **only when** the Store Info `App Icon File` is set (then `manifest.json` also gains an `icons` array); omitted otherwise
- `data/mattresses.json` — regenerated by `build-data.ps1`, which the converter runs. **This generated file — not the CSV — is what the live app reads**, so it must be built by the **modern** `build-data.ps1` (the one in this repo), which preserves the authored copy a stale/old script would strip: `archetype`, `topPickReason` / `topPickReason_es`, and the gold/silver/bronze **tier groupings**. As of **Phase A** the converter treats a **skipped or failed** `build-data.ps1` **as a blocking error by default** — it refuses to emit a deploy-ready bundle whose `mattresses.json` was not regenerated (otherwise a clone would

DreamFinder Build Runbook (Internal)
Page 2 of 15

silently ship the *template's* — i.e. another retailer's — lineup). Pass `--skip-build-json` only for temp/audit output.

Image conventions (handled automatically by the converter):

- Source images may be any common format; output is **always JPG** (quality 88, 1000px long-edge cap, **no WebP**).
- Mattress output filename = `lower(name).jpg` (matches `build-data.ps1`'s lookup) — e.g. a mattress named `Athena` resolves `images/mattresses/athena.jpg`.
- Accessory output filename = the **Image File Name** basename from the Accessories tab (not the accessory id — e.g. `pillow-copper-regular` may map to `copper-ice-regular.jpg`).
- ⚠️ **The Accessories `Image File Name` cell must hold the FULL relative runtime path `images/accessories/<filename>.jpg` — not** a bare filename like `copper-ice-regular.jpg`. `index.html` renders the accessory `image` value **verbatim**, so a bare filename resolves at the site root and **404s live** (this exact bug surfaced during the TFM migration). As of **Phase A**, validation **blocks** a bare filename, a non-`.jpg` extension, or any extra sub-path under `images/accessories/`. The *source* image you drop in the `accessories/` folder is still just `<filename>.jpg` (any format) — only the **cell** carries the full path.
- **Brand logos** are copied **verbatim** (not re-encoded — a transparent PNG stays transparent on the dark footer) from the `brands/` source subdir into `images/brands/`, matched by the **Logo File Name** column (exact filename). Provide each brand logo named exactly as that column value.
- **Not** handled by the converter: the store logo (today it is a **text wordmark** from `logo.main` / `logo.sub`, not an image) and the PWA icons. Those remain manual — see Phase 4.

Validation (runs by default). The converter validates the workbook *before* writing anything and the generated bundle *after*, and **aborts without writing on any blocking error**. To validate a workbook on its own (writes nothing):

```
python tools/validate_workbook.py incoming/Acme_Store_Data.xlsx --source-images
incoming/images
```

As a final readiness gate, treat warnings as blocking and require the GAS URL:

```
python tools/validate_workbook.py incoming/Acme_Store_Data.xlsx --source-images
incoming/images \
  --warnings-as-errors --require-gas-url
```

Converter flags: `--validate-only` (validate, write nothing), `--no-validate` (emergency/debug only), `--warnings-as-errors`, `--require-gas-url`. Validation covers workbook structure, Store Info / config, catalog rows (mattresses, accessories, SalesNotes), source product images, and the generated bundle files. It does **not** replace human review of copy and design.

Verify the toolchain, then smoke-test the bundle locally:

DreamFinder Build Runbook (Internal)
Page 3 of 15

```
python tests/golden/run_golden.py --strict        # toolchain regression (see note)
python -m http.server 8000                         # open http://localhost:8000/
```

> *`run_golden.py` verifies the **Bel** workbook still round-trips to the committed Bel bundle with zero diffs — i.e. the converter/toolchain is healthy. It does **not** judge the new retailer's data. The retailer's bundle is verified by the **local smoke test + manual review** (Phase 7).*

Still manual (the converter does not produce these): the Apps Script / `Code.gs` deploy + GAS `/exec` URL (Phase 5); brand logos, store logo, PWA icons (Phase 4); the live GitHub Pages deploy (Phase 6) and the post-deploy `allowed-hosts.js` 200 + live-load checks (Phases 3 / 7).

Out of scope for this pipeline: Code.gs rewrites, a workbook validation framework (not yet built), and dead-code/CSS cleanup.

Required runtime bundle — completeness checklist

A live deployment is **self-contained**: every file below must be present in the deployed repo. The converter (Phase 2) generates most of them, but three are **shared template files that ride along only because you cloned the repo** (Pattern A) — they are *not* emitted by the converter, so a conversion into a non-clone directory would be missing them. Verify the whole set before cutover:

File / folder	Source
`index.html`	**shared template (from the clone)** — not generated
`data/dict-en.json`	**shared template (from the clone)** — not generated
`data/dict-es.json`	**shared template (from the clone)** — not generated
`data/store-config.json`	generated (converter)
`data/allowed-hosts.js`	generated (converter)
`data/accessories.json`	generated (converter)
`data/mattresses.csv`	generated (converter)
`data/mattresses-es.csv`	generated (converter; omitted if no Spanish copy)

DreamFinder Build Runbook (Internal)
Page 4 of 15

File / folder	Source
`data/mattresses.json`	generated (`build-data.ps1`)
`manifest.json`	generated (converter)
`images/mattresses/`	generated (converter, `--source-images`)
`images/accessories/`	generated (converter, `--source-images`)
`images/brands/`	generated (converter, `--source-images`)
`apple-touch-icon.png` (+ `icon-192.png` , `icon-512.png`)	generated **only when** `App Icon File` is set

> *`index.html` fetches `data/store-config.json` , `data/mattresses.json` , `data/accessories.json` , and `data/dict-<lang>.json` at load, and loads `data/allowed-hosts.js` synchronously before its domain-lock check. A missing `mattresses.json` / `store-config.json` throws on load; a missing `dict-en.json` breaks the UI text; a missing `allowed-hosts.js` blanks the production host. None of these three shared files are validated by the converter, so eyeball them here.*

Phase 3 — Retailer values (generated, plus a few manual)

Most retailer values are now generated by the converter (Phase 2) from the workbook — you do NOT hand-edit `data/store-config.json` , `data/mattresses.*` , `data/accessories.json` , `manifest.json` , or `data/allowed-hosts.js` . Fill the values in the **workbook**, re-run the converter, and they flow into the bundle. The references below document where each value lives / which workbook cells drive it.

`index.html` — no per-store edit needed

`index.html` is fully store-agnostic. In particular, the **domain lock is now config-driven**: the retailer's host goes in the workbook's `allowedHosts` , the converter projects it into `data/allowed-hosts.js` (`window.__DF_ALLOWED_HOSTS = [...]`), and `index.html` loads that file synchronously and reads it (with a built-in `localhost` / `127.0.0.1` fallback). **Do not edit any `allowed` array in `index.html` .**

> ***Deploy dependency — `data/allowed-hosts.js` .*** *Because `index.html` loads `./data/allowed-hosts.js` before its domain-lock check, that file MUST deploy alongside `index.html` . After deploy, confirm it returns **HTTP 200** at `https://<host>.github.io/DreamFinder/data/allowed-hosts.js` (Bel: `https://beford782.github.io/DreamFinder/data/allowed-hosts.js`). If it's missing / 404 the*

DreamFinder Build Runbook (Internal)
Page 5 of 15

> *production host blanks ("Unauthorized domain") for everyone — while `localhost` still works via the fallback, which can mask the problem.*

CSS theme color is driven from `colors.storePrimary` / `colors.accent` in `data/store-config.json` (generated from the workbook) — no color edits in `index.html`.

`data/store-config.json` (generated from the workbook Store Info tab)

These values come from the workbook's **Store Info** tab; the converter writes them into `data/store-config.json` (do not hand-edit the JSON). They are: `storeName`, `storeKey`, `logo.{main, sub}`, `colors.{storePrimary, storePrimaryLight, storePrimaryGlow, accent}`, `gasUrl` (leave blank until Phase 5), `publicAssetRoot` (retailer's GitHub Pages URL with trailing slash), `brands[]`, `text.*` (English UI copy: `pageTitle`, `metaDescription`, `ogTitle`, `trustSignal`, `heritage`, `emailPrivacy`, `privacyPolicyContact`, `inStockText`, `emailHeader`, `emailSubtext`, `locationLabel`), `voice.*` (welcome-screen copy), `discount.codePrefix`. Spanish equivalents (`text_es`, `voice_es`) and the `languages` array are covered in **Language support** below.

The `locationLabel` field (e.g., `"Houston, TX"`) drives the small header location pill. Set it to the retailer's primary store city/state, or leave blank to hide the pill entirely. The kiosk no longer requests browser geolocation — there's no permission prompt and no third-party reverse-geocode call.

The discount is positioned as a **Savings Pass** — premium "private in-store offer" framing rather than a generic "% off" tease. The customer journey (welcome eyebrow, landing tease, results banner, reveal subtitle, email screen, result email subject + body) all reference the Savings Pass. When onboarding a new retailer, review `voice.subCopyAccent` / `voice_es.subCopyAccent` and the corresponding inline strings in `index.html` (`renderNocturnalLanding` + `renderEmailChrome`) and `Code.gs` (subject + email hero band) to confirm the language matches the retailer's tone — no expiration claim or countdown is shown by default.

Language support (`data/store-config.json`)

The template ships bilingual (English + Spanish) by default. A language toggle appears on the welcome screen and customers can flip the whole app — quiz, profile, results, drawer, accessories, email capture, email body — into Spanish.

To configure per retailer, edit the `languages` array:

```
"languages": ["en", "es"]   // default — toggle visible
"languages": ["en"]          // English-only — toggle hidden
```

If Spanish is enabled, also customize the `text_es` block alongside `text`:

```
"text": {
  "socialProof": "Trusted by Acme customers across Arizona",
  "footer": "© 2026 Acme Mattress. All rights reserved.",
```


DreamFinder Build Runbook (Internal)
Page 6 of 15

```
  ...
},
"text_es": {
  "socialProof": "Confiado por clientes de Acme en todo Arizona",
  "footer": "© 2026 Acme Mattress. Todos los derechos reservados.",
  ...
}
```

See `data/dict-en.json` / `data/dict-es.json` for the UI dictionary (all static strings, shared across retailers — rarely need retailer edits).

Spanish mattress descriptions (optional)

To translate mattress-specific copy (badge chips, highlight lines, per-match reason text) into Spanish, create `data/mattresses-es.csv` alongside `data/mattresses.csv`. Columns:

- `id` (required, matches the English CSV)
- `displayBadges`, `highlight`, `reason_cooling`, `reason_pressureRelief`, `reason_motionIsolation`, `reason_support`, `reason_plush`, `reason_medium`, `reason_firm`, `reason_durability`, `reason_default`

Empty cells are allowed — the app falls back to the English value when a Spanish translation is missing. Run `.\build-data.ps1` after editing the CSV to regenerate `data/mattresses.json` with the Spanish fields merged in as `tags_es`, `highlight_es`, `reasons_es`.

If the retailer doesn't want Spanish, just skip creating this file. The app ignores it cleanly.

`Code.gs`

Open in any text editor and replace: - `'Your DreamFinder Results from Bel Furniture'` → retailer's email subject (Store Info → Email Subject Line) - `'Bel Furniture Sleep Team'` (two places) → retailer's sender name (Store Info → Email Sender Name) - `'Show this email at Bel Furniture to redeem.'` → retailer-specific phrasing - `'Bel Furniture x DreamFinder'` (header) → retailer name - `'Bring this email to your Bel Furniture store'` (footer) → retailer phrasing

`manifest.json`

PWA manifest is per-deployment. Update each field for the new retailer:

- [ ] `name` — `"DreamFinder – <Retailer>"` (shown when the kiosk is installed as an app)
- [ ] `description` — `"Personalized sleep consultation for <Retailer>"`
- [ ] `start_url` — confirm it matches the hosted GitHub Pages path. Default is `"/DreamFinder/"` (matches a repo named `DreamFinder`). If the retailer's repo is named differently (e.g., `acme-dreamfinder`), set `"/<repo-name>/"` instead, or the kiosk will 404 on launch.

DreamFinder Build Runbook (Internal)
Page 7 of 15

- [ ] `theme_color` and `background_color` — confirm these match the retailer's active theme. The Bel template ships `#0f1f33` (a pre-Nocturnal navy); align with the current `--color-bg` if the retailer adopts the Nocturnal palette, or set to the retailer's brand background.

```
{
  "name": "DreamFinder — Acme Mattress",
  "short_name": "DreamFinder",
  "description": "Personalized sleep consultation for Acme Mattress",
  "start_url": "/DreamFinder/",
  "display": "standalone",
  "orientation": "landscape",
  "background_color": "#0f1f33",
  "theme_color": "#0f1f33"
}
```

Phase 4 — Store logo, PWA icons, favicon

The app's favicon is an inline SVG near the top of `index.html` (search for `<link rel="icon"`). It uses the gold moon — leave it as-is unless the retailer wants their own.

The **store logo** is currently a **text wordmark** driven by `logo.main` / `logo.sub` in `store-config.json` (the Logo Line 1 / Logo Line 2 columns on the Store Info tab) — there is no store-logo *image* in the app today. Nothing needs to be dropped in for it.

PWA icons are optional and supported. To give a retailer an installable-app icon:

1. Drop one **square PNG, ≥ 512×512** into the `logos/` image source folder.
2. Put its filename in the **App Icon File** column on the Store Info tab.

When `App Icon File` is set (and you run the converter with `--source-images`), the pipeline generates `icon-192.png`, `icon-512.png`, and `apple-touch-icon.png` (180×180) at the repo root and adds an `icons` array to `manifest.json`. If `App Icon File` is **blank**, no icons are generated and `manifest.json` has no `icons` key — the app works fine without it.

`index.html` includes a static `<link rel="apple-touch-icon" href="apple-touch-icon.png">` so an installed iPad home-screen shortcut shows the app icon. A deployment that does not provide an app icon simply 404s that one link (benign — the inline SVG favicon is unchanged).

Validation guards (added in `8c7254a`). When `App Icon File` is set, the converter will not silently ship a bundle whose icons went missing:

- **Up front (M2):** if you run with `App Icon File` set but **without** `--source-images`, or **with** `--skip-image-normalization`, the converter **fails before writing anything** rather than emit a `manifest.json`

DreamFinder Build Runbook (Internal)
Page 8 of 15

that drops its `icons` array. The source PNG is checked here too — it must be a **square PNG ≥ 512×512** or the run errors.

- **Post-emit (M3):** whenever `manifest.json` declares an `icons` array, validation re-checks the output directory and **errors if `apple-touch-icon.png` is absent**. That file is referenced by `index.html` but is **not** listed in `manifest.icons`, so it is verified explicitly.

A **blank** `App Icon File` skips both guards — an icon-less bundle is valid (no `icons` key, no `apple-touch-icon.png`).

> ***Bel is enabled.*** *Its source icon is `images/logos/app-icon.png` (the gold-moon crescent on `#0f1f33`), `App Icon File` = `app-icon.png`, and `manifest.json` + the three root icons are committed.*

Phase 5 — Set up Google Apps Script for the retailer

Each retailer needs their own GAS web app, sending email from their own Google account and writing to their own Sheet.

1. Create a new Google Sheet under their Google account. Name it `<Retailer> DreamFinder Leads`.
2. Tools → Apps Script. This opens a new project bound to the sheet.
3. Delete the placeholder code. Paste the contents of the repo's `Code.gs` (with the Phase 3 retailer-specific text edits already applied). Then review the `RESULT_EMAIL_BCC` constant near the top of the file: - Default: `'dreamfinderleads@gmail.com'` — the central DreamFinder lead backup inbox. - Replace with a retailer-specific backup inbox if the retailer wants its own backup destination. - Set to `''` only if BCC backup is intentionally disabled for this deployment.
4. Save. Click **Deploy** → **New deployment**.
5. Gear icon → **Web app**.
6. Configure: - Description: `Initial` - Execute as: **Me** (the retailer's account) - Who has access: **Anyone**
7. Click **Deploy**. Authorize when prompted.
8. **Copy the Web app URL** that ends in `/exec`.
9. Paste it into `data/store-config.json` as the `gasUrl` value. Save.

Note: when `Code.gs` is edited later (security patches, copy changes, etc.), the live web app does NOT auto-update. See **Phase 9** below for the redeploy + test-send checklist.

Phase 6 — Stage on a branch, then cut over to main

Branch-first staging (the TFM-proven pattern)

DreamFinder Build Runbook (Internal)
Page 9 of 15

Do **not** push a brand-new retailer straight to `main`. GitHub Pages deploys from `main`, so any push there is a live deploy. Stage first:

1. Create a retailer-specific staging branch (e.g. `acme-redesign`) and commit the generated bundle there.
2. Validate **locally** on that branch (Phase 2 final gate + Phase 7 smoke test).
3. **Push the staging branch only:** `git push origin acme-redesign`. This does *not* deploy (Pages serves `main`).
4. Cut over to `main` **only after explicit sign-off**. Keep the staging branch intact afterward as a reference (TFM kept `tfm-redesign` pointing at its cutover commit).

> ***Remote safety — check before every push.*** *Run `git remote -v` first and confirm `origin` points at this retailer's repo.* ***Never push one retailer's repo to another retailer's remote.*** *In particular, the TheFurnitureMarket clone carries an extra `bel` remote pointing at the DreamFinder (Bel) repo —* ***never push TFM to `bel`****.*

Cut over to main

```
git add .
git commit -m "Initial Acme deployment"
git push origin main
```

(For subsequent deploys this repo uses a force push — `git push origin main --force`, or the `git ship` alias — per the project convention.)

Then in the new repo on GitHub: 1. Settings → Pages 2. Source: **Deploy from a branch** 3. Branch: **main** / folder: **/ (root)** 4. Save

Wait 1–3 minutes for the first deploy. Pages URL will be:

```
https://acmemattress.github.io/DreamFinder/
```

After the deploy, verify the **M1 domain lock** is healthy: - the retailer's host is in the workbook `allowedHosts` (so `data/allowed-hosts.js` lists it), - `https://<host>.github.io/DreamFinder/data/allowed-hosts.js` returns **HTTP 200**, and - the live site loads on the production host (not just localhost).

A missing `data/allowed-hosts.js` blanks the production host — see the Phase 3 deploy-dependency note.

Rollback (GitHub Pages)

Before cutting over, record the current good `main` SHA: `git rev-parse origin/main`. If the new deploy turns out bad, roll `main` back to that SHA:

DreamFinder Build Runbook (Internal)
Page 10 of 15

```
git push origin <prior-good-sha>:main
```

Then wait 1–3 minutes for GitHub Pages to redeploy and **verify on the production host**: load the page, confirm `data/allowed-hosts.js` returns HTTP 200, and check the console is clean. (TFM example: the recorded prior-good SHA was `4c8d3df`, so its rollback command was `git push origin 4c8d3df:main`.)

Phase 7 — End-to-end verification

Open the Pages URL on a desktop browser AND on an iPad. Run the full flow:

- [ ] Welcome screen shows the retailer's name, colors, trust signal, badge
- [ ] Quiz runs all 12 questions; skip-logic works for solo sleepers; multi-select cap of 3 enforced
- [ ] Review screen shows current answers; Edit returns to that question
- [ ] Profile screen shows the personalized opener (uses `trigger`, `current_mattress_age`, `sleep_quality`)
- [ ] Confidence pill + social proof show
- [ ] Theme accent colors look right per profile archetype
- [ ] Results page: Gold/Silver/Bronze tabs work; tier explainer reads correctly; $/$$/$$$ price markers visible
- [ ] Mattress card opens drawer; drawer prev/next works; mattress images load
- [ ] Compare 2 button works; compare modal renders side-by-side
- [ ] Accessories flow works — "Build Your Sleep System" CTA takes customer through foundations → pillows → protectors
- [ ] **Every accessory image loads** — foundations, pillows, and protectors all show their photo, with **no broken white boxes / missing-image placeholders**. A broken accessory image almost always means a bare `Image File Name` cell (it must be the full `images/accessories/<file>.jpg` path — see Phase 2)
- [ ] **Adjustable base hero appears** when quiz includes snoring, reflux, or back pain — shows personalized benefit cards and animated base SVG
- [ ] **Hero "Continue to Foundations" button** advances the flow (no auto-scroll)
- [ ] **Featured top pillow** has gold border + "Matched to Your Profile" badge + 2 personalized reasons
- [ ] **Protector "Did You Know?" callout** shows between the section header and product cards
- [ ] Pillow step intro text is personalized to sleep position (side/back/stomach)
- [ ] Email submit:
- [ ] No DEBUG row in the sheet
- [ ] Single clean row written: date, name, email, phone, DREAM code, matches, accessories
- [ ] Customer email arrives with mattress images + DREAM code visible
- [ ] Email reflects the customer's saved picks, not just algorithm picks

DreamFinder Build Runbook (Internal)
Page 11 of 15

- [ ] Restart button works from header, accessories screen, and email screen
- [ ] Discount overlay doesn't block taps; full reveal completes before fade-out
- [ ] Idle timer (2 min) returns to welcome
- [ ] **Spanish toggle (EN/ES) visible on welcome screen**
- [ ] **Tap ES — entire welcome screen switches to Spanish**
- [ ] **Complete quiz in Spanish — all 12 questions, labels, sublabels in Spanish**
- [ ] **Profile, results, drawer, accessories all render in Spanish**
- [ ] **Submit email in Spanish mode — email arrives with Spanish subject, body, labels**
- [ ] **Tap EN — reverts to English cleanly**
- [ ] **Idle reset returns language to English**
- [ ] Console (DevTools) is clean — **no 404s** (watch especially for `images/accessories/*.jpg`) and no red errors
- [ ] **Verify on the live production host, not just `localhost`** — the domain-lock `localhost` / `127.0.0.1` fallback masks a missing/blank `data/allowed-hosts.js`, so a localhost-only pass can hide an "Unauthorized domain" failure (and other path issues) that only surface on the real Pages host

---

Phase 8 — Hand-off

- [ ] Send the retailer their live Pages URL
- [ ] Walk a salesperson through the kiosk flow, including the handoff screen with the sales cheat sheet
- [ ] Confirm they can access the lead Sheet
- [ ] Schedule a 1-week check-in: was everything submitted as expected, any odd lead patterns, etc.
- [ ] Add the new retailer's repo to your bookmark list

Phase 9 — Updating Apps Script after `Code.gs` changes

`Code.gs` does not auto-deploy. After repo edits (security hardening, sender-name changes, etc.), promote manually.

Redeploy checklist

1. Open the bound Apps Script editor (Sheet → Extensions → Apps Script).
2. Select `Code.gs` in the file panel; delete its existing contents.
3. Paste the updated `Code.gs` from this repo (`cat Code.gs | clip` from the repo root, then Ctrl+V in the editor).
4. **Save** (Ctrl+S). No syntax errors should appear.

DreamFinder Build Runbook (Internal)
Page 12 of 15

5. **Deploy** → **Manage deployments**. Click the **pencil icon** next to the existing web-app deployment.
6. Version dropdown → **New version**. Add a short description of what changed. Click **Deploy**.
7. Verify the resulting **Web app URL matches** `gasUrl` in `data/store-config.json`. If it differs, you accidentally created a *new* deployment instead of a new version — revert and retry.

Post-deploy test send

Run from the live kiosk (or a desktop browser at the Pages URL). Cover at least:

- [ ] **Normal flow** — typical name + email, EN. Email arrives; Sheet row appended in the locked column order (`timestamp, name, email, phone, dreamCode, lang, matches, accessories, rsa`).
- [ ] **Invalid email** — submit empty or malformed (e.g., `abc` , whitespace, missing `@`). GAS returns `{success:false, error:"invalid_email"}` . Sheet row NOT appended. No email sent.
- [ ] **Punctuation / accented name** — `O'Connor` , `Bel-O-Pedic` , `Ñoño` . Apostrophes / hyphens / accents preserved verbatim in email body and Sheet row.
- [ ] **Injection-looking name** — submit something like `O'Connor Ñoño <script>alert(1)</script>` . Email body shows the literal text including angle brackets; no script execution. Validates HTML escape coverage.
- [ ] **Spanish flow** (if `languages` includes `"es"`) — switch to ES on the welcome screen, complete in Spanish, submit. Subject + body in Spanish. Sheet `lang` column reads `es` .

If any check fails, revert via **Manage Deployments** → **pencil** → **Version** → **[previous]** → **Deploy**.

Per-retailer files to keep as artifacts

After delivering, archive a copy of: - The completed `Acme_Store_Data.xlsx` (the source of truth) - The retailer's `Code.gs` (in case you need to redeploy after future engine changes) - The Pages URL + GAS Web app URL + Sheet URL in a shared "Retailers" doc

This makes future changes (engine version bumps, scoring tweaks) a re-run rather than a re-build.

Regenerating the onboarding PDFs / kit

The PDFs in this folder (`DreamFinder_Onboarding_Guide.pdf` , `DreamFinder_Image_Upload_Guide.pdf` , and the internal `DreamFinder_Build_Runbook.pdf`) are generated from the source `.md` / `.txt` files here by `tools/md_to_pdf.py` . After editing a source doc, regenerate:

```
python tools/md_to_pdf.py        # builds all three PDFs into onboarding/
```


DreamFinder Build Runbook (Internal)
Page 13 of 15

Toolchain / backends. `md_to_pdf.py` has **two interchangeable backends** that produce visually equivalent PDFs; the backend is auto-selected (WeasyPrint if importable, else Edge) and can be overridden with `--backend` or the `DF_PDF_BACKEND` environment variable.

- **Edge (Windows-native — the supported Windows path).** Drives headless Microsoft Edge (or Chrome) over the DevTools protocol using only the Python standard library, so just `pip install markdown` and run. **No GTK runtime and no WSL required:**

```
python tools/md_to_pdf.py # auto-selects Edge when weasyprint is absent python tools/md_to_pdf.py --backend edge
```

- **WeasyPrint (Linux / WSL / macOS — optional).** The original path. It loads a **native GTK / Pango / Cairo runtime** at import, so `pip install weasyprint` is **not** enough on Windows — there it fails with `cannot load library 'libgobject-2.0-0'`. Use it only when you specifically want the WeasyPrint renderer, e.g. from WSL / Linux:

```
sudo apt install libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz0b pip install markdown weasyprint python tools/md_to_pdf.py --backend weasyprint
```

Keep using the same backend that produced the committed PDFs (Edge) unless you intend to switch — the two render slightly differently, so swapping backends creates large binary diffs for no content change.

Then rebuild `DreamFinder_Retailer_Onboarding_Kit.zip` with exactly the guide PDF, the image-upload PDF, and the current blank template (`DreamFinder_Onboarding_Template.xlsx`) — **not** the internal runbook PDF.

> *The committed PDFs / zip can lag the source docs. Regenerate them before handing onboarding materials to a retailer; until then, the `.md` / `.txt` sources in this folder are authoritative.*

Common gotchas

- **GAS Web app URL changed silently after a redeploy.** If you create a *new* deployment instead of updating the existing one, the URL changes. Always use **Deploy → Manage deployments → pencil → New version**.
- **Mattress images broken in the customer email.** The client converts relative URLs to absolute using the `publicAssetRoot` field in `data/store-config.json` — make sure that field points at the new retailer's Pages URL with a trailing slash (e.g., `https://acmemattress.github.io/DreamFinder/`). Operators edit the JSON, not a constant in the code.
- **"Unauthorized domain" splash on first load.** The host isn't in the M1 allowlist. Set the retailer's Pages host in the workbook's `allowedHosts` , regenerate the bundle (which produces `data/allowed-`

DreamFinder Build Runbook (Internal)
Page 14 of 15

`hosts.js`), and commit + deploy that file. Do **not** edit any `allowed` array in `index.html`. After deploy, confirm `https://<host>.github.io/DreamFinder/data/allowed-hosts.js` returns **HTTP 200** — a missing file blanks the production host (localhost still passes via the fallback, so it can mask the problem).

- **DEBUG rows reappear in the lead Sheet.** That's an old GAS deployment still serving traffic. Confirm `data/store-config.json` `gasUrl` matches the live Apps Script deployment URL exactly.
- **Mattress images broken in Outlook desktop / iOS Mail.** This is why the converter emits **JPG only** (no WebP) — Outlook's Word render engine and iOS Mail handle WebP/PNG unreliably. If you see broken email images, check that `images/` holds the converter's JPGs (re-run Phase 2 if any legacy `.webp` lingers); don't hand-place WebP product images.

DreamFinder Build Runbook (Internal)
Page 15 of 15